

Electric Vehicle Market Segmentation Analysis: Valued EV Selection

By

Kumar Naga Sai Siva Shankar Ganireddi

Introduction

This project focuses on segmenting the electric vehicle (EV) market using various clustering techniques to understand how different EV attributes influence market trends. By analyzing price, range, powertrain type, and customer behaviors, this analysis identifies valued EV models based on their performance and cost.

Main Objectives:

1. Market Segmentation: Segment EVs based on attributes such as price, range, and speed.
2. Behavioral Analysis: Explore customer behaviors related to EV purchase decisions.
3. Valued EV Selection: Use clustering methods to select the most valuable EVs based on customer needs and price trends.

Step 1: Data Exploration & Cleaning

In this phase, we load and explore the dataset that contains EV attributes such as price, range, powertrain, top speed, etc. Cleaning tasks involve checking for missing values and duplicate rows to ensure accurate data for analysis.

Key Highlights:

Data Overview: The dataset includes fields like EV price, range, efficiency, top speed, and powertrain type.

Data Cleaning: Missing and duplicate values are identified and handled to maintain data integrity.

Step 2: Visualization of Key EV Metrics

Data visualization helps to identify trends in the EV market. By plotting features like price, range, top speed, and powertrain, we gain valuable insights into how these attributes influence EV selection.

Key Insights:

1. Scatter Plot of Price Distribution: EV prices are distributed across a wide range, with a few outliers indicating premium models.
2. Price vs Range: A line chart reveals how EV prices correlate with the driving range.
3. Price by PowerTrain: A violin plot highlights the differences in pricing between various powertrain types.
4. Top Speed by Brand: Bar plots show which brands offer the highest top speeds, indicating performance-focused EV models.
5. Plug Type Distribution: A doughnut chart displays the dominance of certain plug types in the EV market, critical for infrastructure planning.

Step 3: Clustering the EV Market

To group EVs into different market segments, K-Means clustering is applied to segment the data based on key attributes like price, top speed, and range. The elbow method is used to determine the optimal number of clusters, ensuring meaningful segmentation.

K-Means Clustering:

Elbow Method: Identifies the ideal number of clusters by plotting the within-cluster sum of squares (WCSS) to determine when adding more clusters doesn't significantly improve the fit.

Cluster Analysis: Vehicles are clustered based on speed and range, helping to categorize EVs into economy, mid-range, and premium segments.

Step 4: Behavioral Analysis on Customer Data

A separate dataset related to customer behavior (e.g., car loan status, salary, dependents) is analyzed. This dataset helps to explore how demographic and financial factors influence customer preferences for electric vehicles.

Behavioral Insights:

Car Loan Status: Visualized through a doughnut chart, showing how many customers require loans for their EV purchase.

Demographic Trends: Plots showing how age, number of dependents, and salary influence EV purchasing decisions.

Step 5: K-Prototypes Clustering on Customer Data

K-Prototypes clustering, which handles both categorical and numerical data, is applied to group customers into segments based on features such as age, salary, car loan status, and number of dependents. This segmentation helps to identify distinct customer groups, offering insights into which clusters are likely to opt for specific EV models.

Customer Segmentation:

Elbow Method: Determines the optimal number of clusters, balancing the need for sufficient segmentation without overcomplicating the model.

Customer Clusters: Each cluster provides a distinct view of customer behavior, allowing for targeted marketing strategies.

Step 6: Valued EV Selection

By cross-referencing the results from EV clustering with customer segments, we are able to identify valued EV selections that cater to specific customer needs. For example, clusters with lower income and loan dependency may prefer EVs from the economy segment, while higher-income clusters may focus on premium models.

Key Findings:

EV Price vs Age: Younger customers often opt for more affordable EVs, whereas older customers may lean toward more expensive models.

EV Price vs Salary: Higher salary correlates with a preference for premium EVs, while mid-range EVs appeal to the broader market.

EV Price vs Number of Dependents: Families with more dependents prefer affordable EVs, indicating a need for budget-friendly electric vehicles with longer ranges.

Conclusion

This analysis provides an in-depth understanding of the EV market and customer behavior, leading to the selection of valuable EV models based on price, performance, and customer needs. By segmenting the market through clustering techniques, businesses can identify which EV models are likely to succeed in different market segments, offering more targeted EV options for consumers.

✓ Electric Vehicle Market Segmentation Analysis Focusing on Valued EV selection

This notebook performs an in-depth analysis and visualization to understand the market segmentation of electric vehicles (EVs). The aim is to explore the dataset, identify key trends, and apply clustering techniques to segment the EV market.

✓ Step 1: Import Necessary Libraries

We start by importing the essential libraries required for data manipulation, visualization, and clustering analysis.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Suppress warnings for cleaner output
import warnings
warnings.filterwarnings("ignore")

import warnings
warnings.filterwarnings("ignore")
```

✓ Step 2: Load the Dataset

We load the dataset that contains various attributes of electric vehicles, which will be used for further analysis and segmentation.

```
# Load the dataset
df = pd.read_csv("Electric-car-data.csv")
```

✓ Step 3: Explore the Data

Explore the dataset by displaying the first few rows, checking data types, and generating summary statistics.

```
# Display the first 10 rows of the dataset
df.head(10)

# Display information about the dataset
print(df.info())

# Display summary statistics
print(df.describe())

# Check the shape of the dataset
df.shape

# Select a subset of columns for initial exploration
ilocfunc = df.iloc[:,0:9]
ilocfunc
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103 entries, 0 to 102
Data columns (total 14 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   Brand               103 non-null    object  
 1   Model               103 non-null    object  
 2   AccelSec            103 non-null    float64 
 3   TopSpeed_KmH       103 non-null    int64   
 4   Range_Km           103 non-null    int64   
 5   Efficiency_WhKm     103 non-null    int64   
 6   FastCharge_KmH     103 non-null    object  
 7   RapidCharge        103 non-null    object  
 8   PowerTrain         103 non-null    object  
 9   PlugType           103 non-null    object  
10   BodyStyle          103 non-null    object  
11   Segment            103 non-null    object  
12   Seats              103 non-null    int64   
13   PriceINR           103 non-null    int64   
dtypes: float64(1), int64(5), object(8)
memory usage: 11.4+ KB
None

   count  AccelSec  TopSpeed_KmH  Range_Km  Efficiency_WhKm  Seats \
mean    7.396117  179.194175    338.786408    189.165049    4.883495
std     3.017430    43.573030    126.014444    29.566839    0.795834
min     2.100000    123.000000    95.000000    104.000000    2.000000
25%     5.100000    150.000000    250.000000    168.000000    5.000000
50%     7.300000    160.000000    340.000000    180.000000    5.000000
75%     9.000000    200.000000    400.000000    203.000000    5.000000
max    22.400000    410.000000    970.000000    273.000000    7.000000

   PriceINR
count  1.030000e+02
mean   5.134664e+06
std    3.140389e+06
min    1.851868e+06
25%    3.167514e+06
50%    4.140000e+06
75%    5.980000e+06
max    1.978000e+07

```

	Brand	Model	AccelSec	TopSpeed_KmH	Range_Km	Efficiency_WhKm	FastCharge_KmH	RapidCharge	PowerTrain
0	Tesla	Model 3 Long Range Dual Motor	4.6	233	450	161	940	Yes	AWD
1	Volkswagen	ID.3 Pure	10.0	160	270	167	250	Yes	RWD
2	Polestar	2	4.7	210	400	181	620	Yes	AWD
3	BMW	iX3	6.8	180	360	206	560	Yes	RWD
4	Honda	e	9.5	145	170	168	190	Yes	RWD
...	...	...	...	...	...	...	...	...	...
98	Nissan	Ariya 63kWh	7.5	160	330	191	440	Yes	FWD
99	Audi	e-tron S Sportback 55 quattro	4.5	210	335	258	540	Yes	AWD
100	Nissan	Ariya e-4ORCE 63kWh	5.9	200	325	194	440	Yes	AWD
101	Nissan	Ariya e-4ORCE 87kWh Performance	5.1	200	375	232	450	Yes	AWD
102	Hyundai	IONIQ 6 77.7 kWh AWD	7.5	190	400	238	480	Yes	AWD

Step 4: Data Cleaning

Check for missing or duplicate values in the dataset and address them before proceeding with further analysis.

```

# Check for missing values in the dataset
df.isnull().sum()

# Check for duplicate rows in the dataset
df.duplicated().sum()

# Display column names
df.columns

```

```
Index(['Brand', 'Model', 'AccelSec', 'TopSpeed_KmH', 'Range_Km',
      'Efficiency_WhKm', 'FastCharge_KmH', 'RapidCharge', 'PowerTrain',
      'PlugType', 'BodyStyle', 'Segment', 'Seats', 'PriceINR'],
      dtype='object')
```

Step 5: Data Visualization

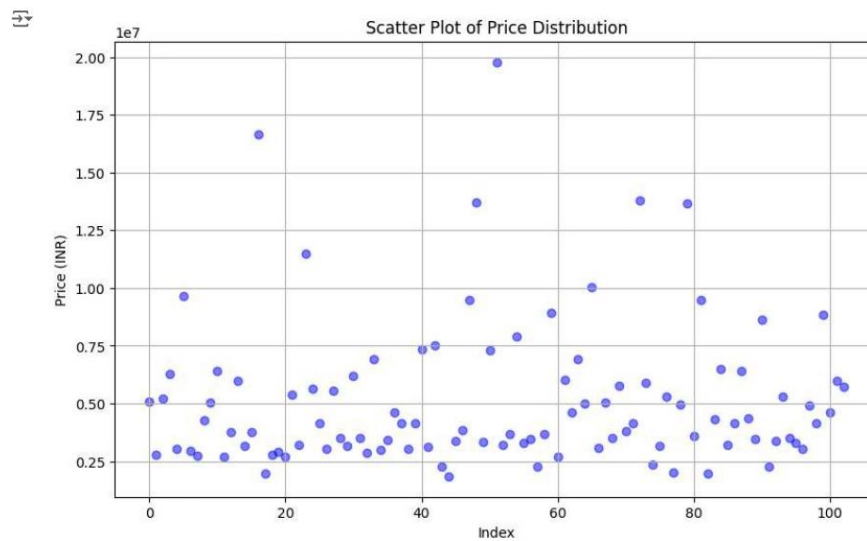
Visualize the data using various plots to identify key trends and insights.

Scatter Plot of Price Distribution

```
# Adjust the size of the plot
plt.figure(figsize=(10, 6))

# Create scatter plot
plt.scatter(range(len(df)), df["PriceINR"], color='blue', alpha=0.5)
plt.xlabel("Index")
plt.ylabel("Price (INR)")
plt.title("Scatter Plot of Price Distribution")
plt.grid(True)

plt.show()
```

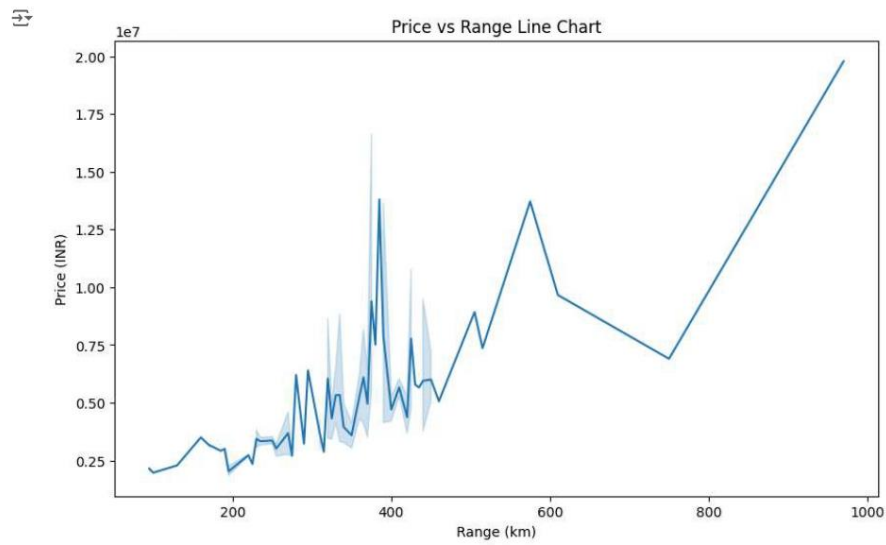


Line Chart of Price vs. Range

```
# Adjust the size of the plot
plt.figure(figsize=(10, 6))

# Create line chart
sns.lineplot(x="Range_Km", y="PriceINR", data=df)
plt.xlabel("Range (km)")
plt.ylabel("Price (INR)")
plt.title("Price vs Range Line Chart")

plt.show()
```

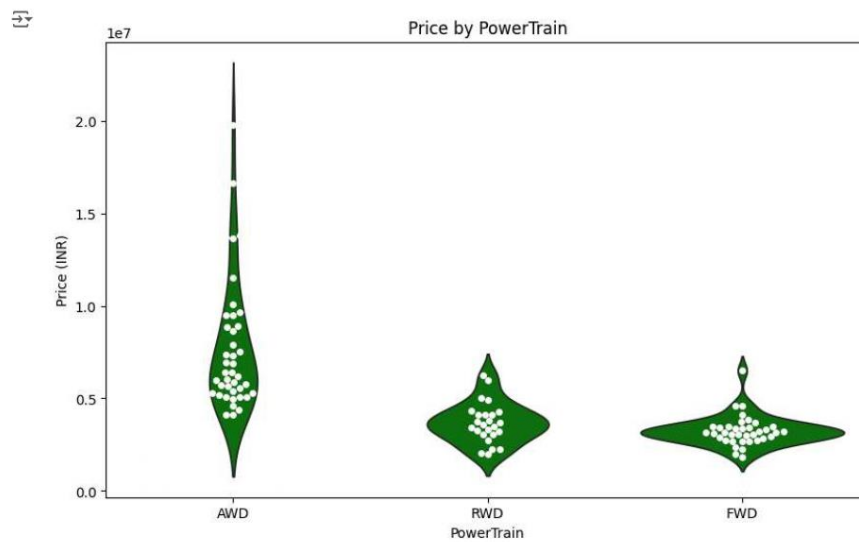


Violin Plot of Price by Power Train

```
# Adjust the size of the plot
plt.figure(figsize=(10, 6))

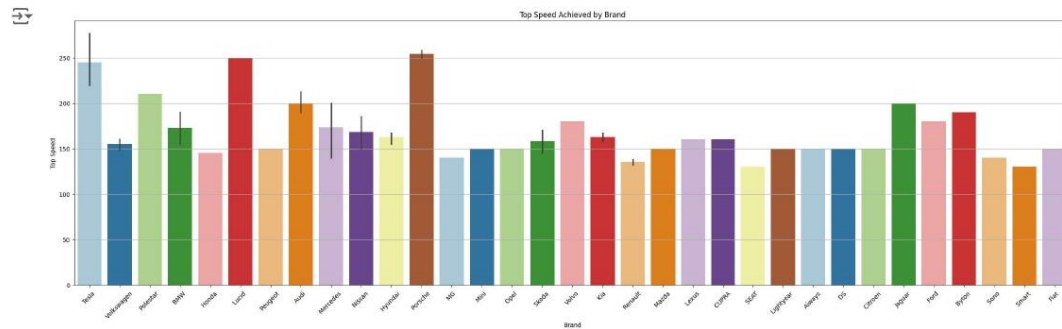
# Create violin plot
sns.violinplot(x="PowerTrain", y="PriceINR", data=df, color="green", inner=None)
sns.swarmplot(x="PowerTrain", y="PriceINR", data=df, color="white", edgecolor="gray")
plt.title('Price by PowerTrain')
plt.xlabel('PowerTrain')
plt.ylabel('Price (INR)')

plt.show()
```



Bar Graph of Top Speed by Brand

```
# Adjust the figure size
plt.figure(figsize=(30, 8))
sns.barplot(x='Brand', y='TopSpeed_KmH', data=df, palette='Paired')
plt.grid(axis='y')
plt.title('Top Speed Achieved by Brand')
plt.xlabel('Brand')
plt.ylabel('Top Speed')
plt.xticks(rotation=45)
plt.show()
```



Doughnut Chart for Plug Type

```
# Import necessary library for plotting
import matplotlib.pyplot as plt

# Adjust the figure size
plt.figure(figsize=(8, 8))

# Create pie chart
plt.pie(df['PlugType'].value_counts(), labels=df['PlugType'].value_counts().index, autopct='%0f%%', explode=(.1, .1, .1, .1))

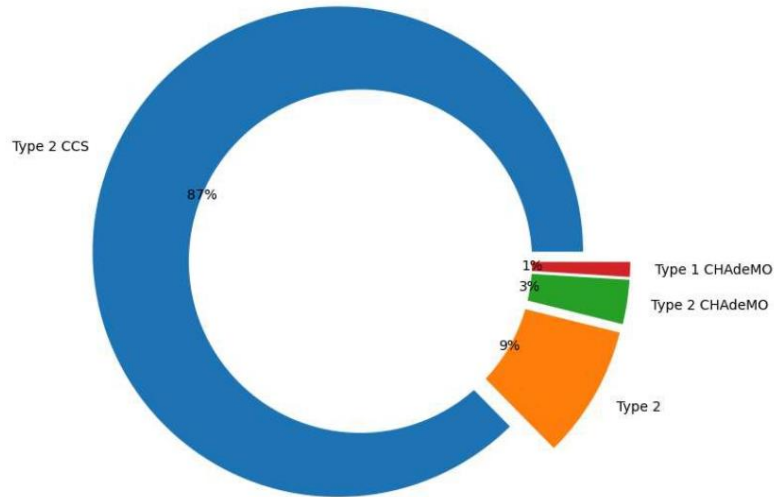
# Draw a white circle in the center to create a doughnut chart effect
centre_circle = plt.Circle((0, 0), 0.70, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.title('Plug Type')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle

plt.tight_layout() # Adjust layout for better spacing
plt.show()
```



Plug Type

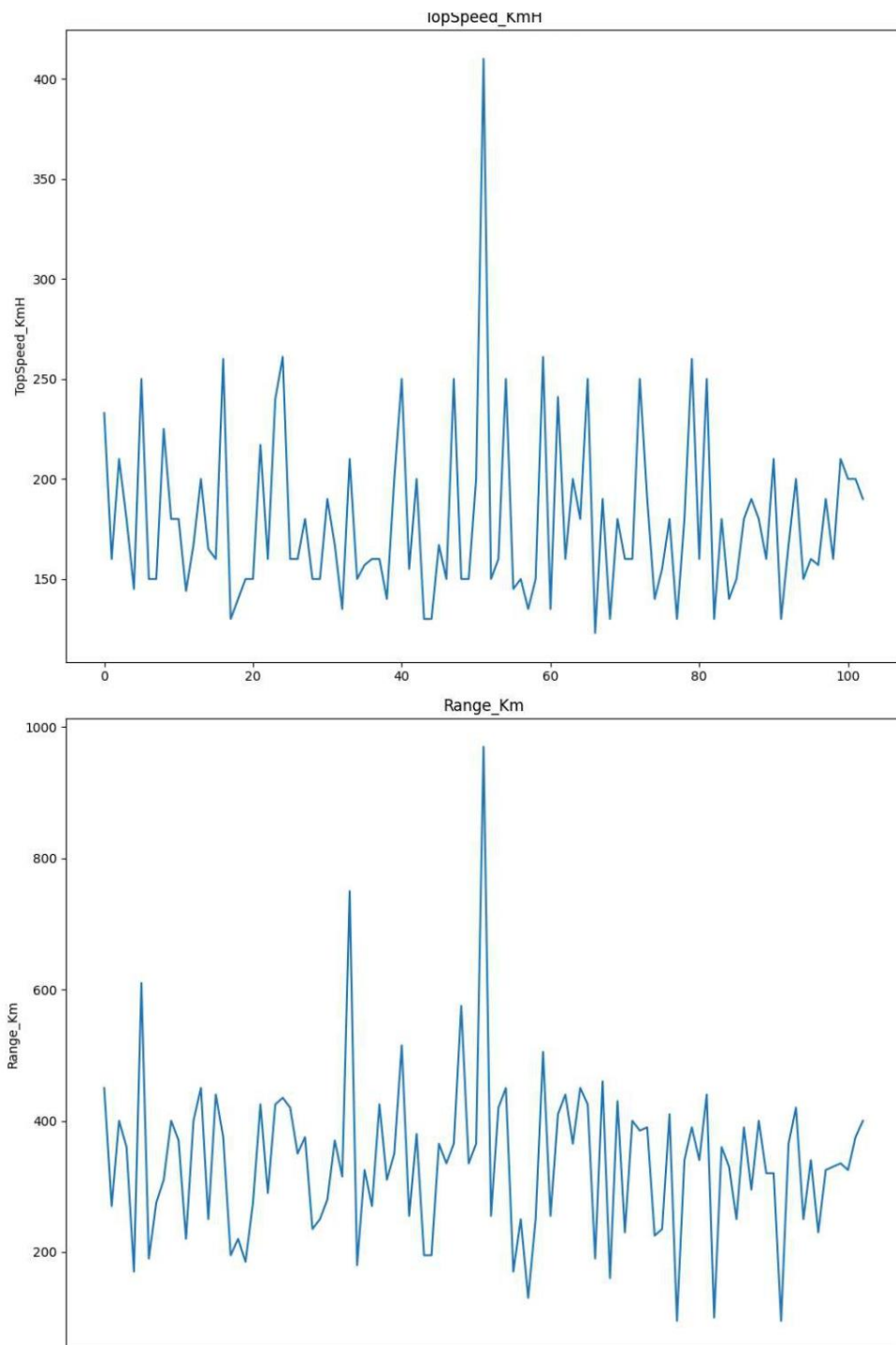


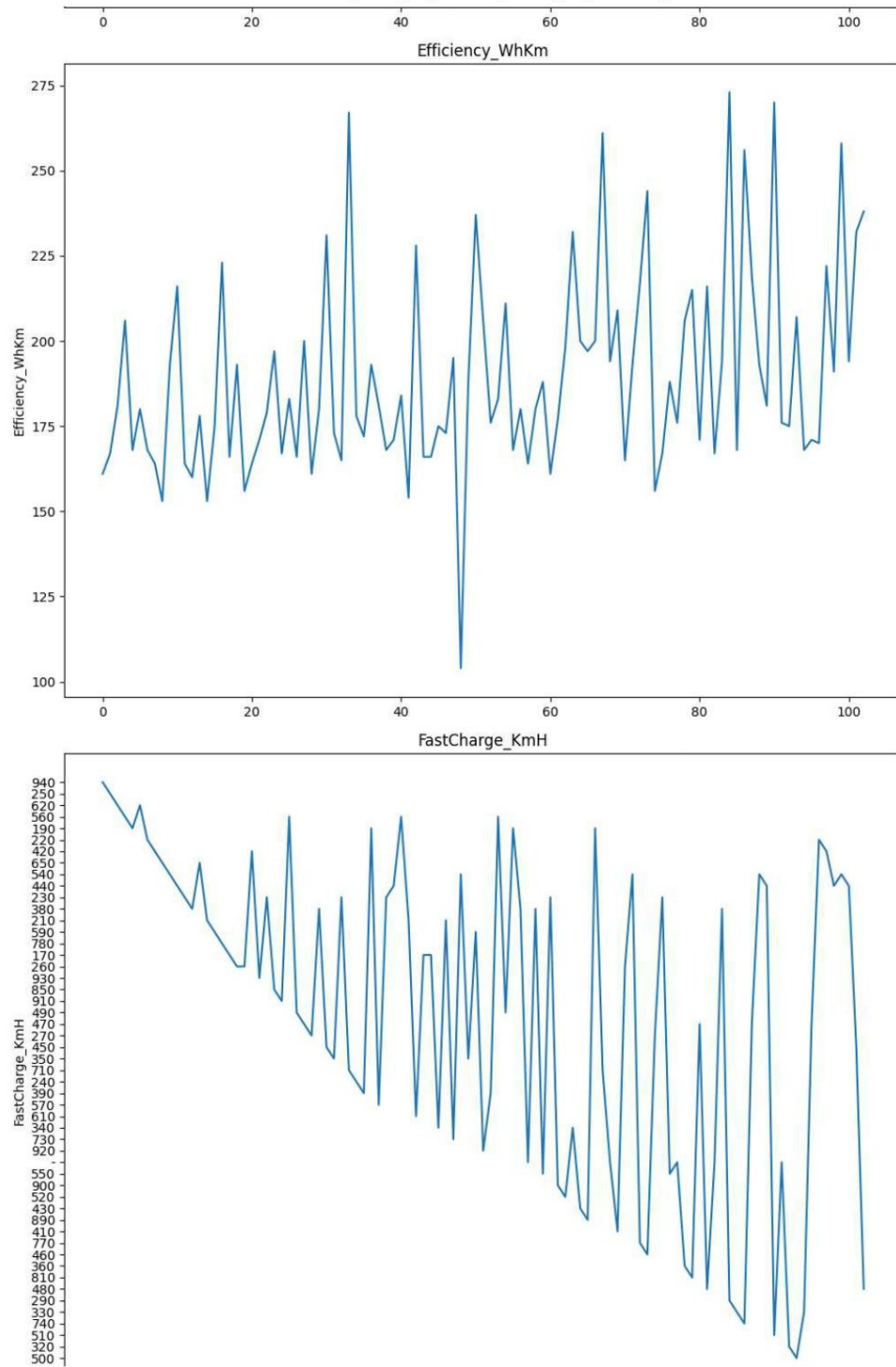
Line Charts for Various Features

```
# Adjust the figure size
plt.figure(figsize=(10, 30))
n = 0

# Plot line charts for various features
for i, x in enumerate(['TopSpeed_KmH', 'Range_Km', 'Efficiency_WhKm', 'FastCharge_KmH'], 1):
    plt.subplot(4, 1, i)
    sns.lineplot(data=df[x])
    plt.title(f'{x}')
    plt.xlabel('') # Remove x-axis label for better layout

plt.tight_layout() # Adjust spacing between subplots
plt.show()
```



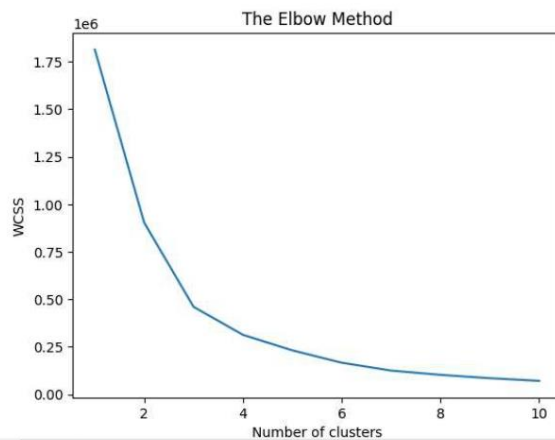


Step 6: K-Means Clustering

Apply K-Means clustering to segment the data based on selected features.

```
# Extracting relevant features for clustering
X = df.iloc[:, [3, 4]].values

# Using the elbow method to find the optimal number of clusters
from sklearn.cluster import KMeans
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



Step 7: Convex Clustering

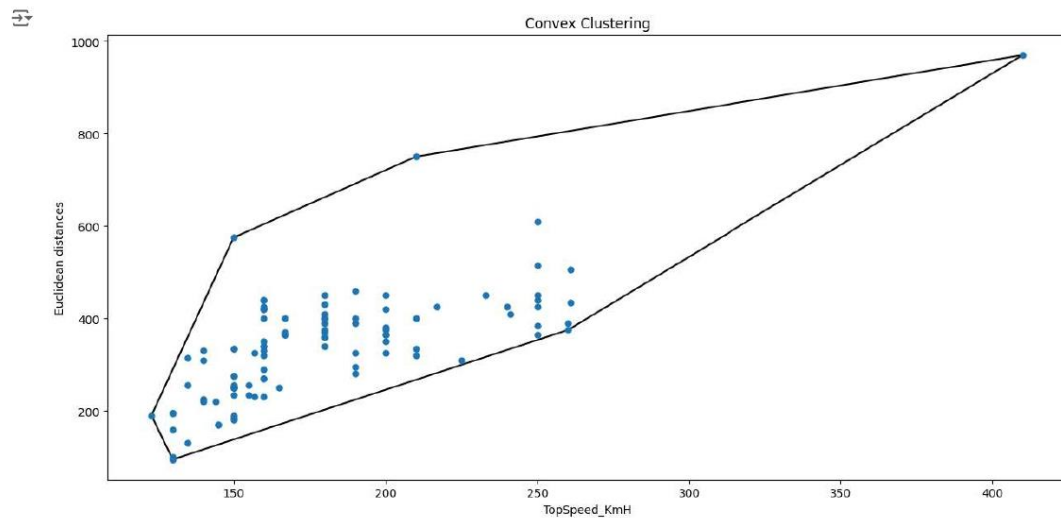
Perform convex clustering and visualize the results.

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.spatial import ConvexHull

# Perform convex clustering
hull = ConvexHull(X)

# Plot the convex hull
plt.figure(figsize=(15, 7))
for simplex in hull.simplices:
    plt.plot(X[simplex, 0], X[simplex, 1], 'k-') # Plot the edges of the convex hull

plt.plot(X[:, 0], X[:, 1], 'o', markersize=5) # Plot the data points
plt.title('Convex Clustering')
plt.xlabel('TopSpeed_KmH')
plt.ylabel('Euclidean distances')
plt.show()
```



✓ Step 8: Loading and Analyzing New Dataset

Load and analyze a new dataset to understand customer behavior.

```
# Import necessary libraries
import plotly.express as px
!pip install kaleido
import kaleido

# Load the new dataset
data = pd.read_csv("behavioural_dataset.csv")

# Describe the dataset
data.describe()

# Check for missing values
print(pd.isnull(data).sum())

# Rename columns for clarity
data.rename(columns={'Personal loan': 'Car_loan'}, inplace=True)
data.rename(columns={'Price': 'EV_Price'}, inplace=True)
data.head()
```

```
Collecting kaleido
  Downloading kaleido-0.2.1-py2.py3-none-manylinux1_x86_64.whl.metadata (15 kB)
  Downloading kaleido-0.2.1-py2.py3-none-manylinux1_x86_64.whl (79.9 MB)
    79.9/79.9 MB 12.3 MB/s eta 0:00:00
Installing collected packages: kaleido
Successfully installed kaleido-0.2.1
Age      0
Profession  0
Marrital Status  0
Education  0
No of Dependents  0
Personal loan  0
Total Salary  0
Price      0
dtype: int64
```

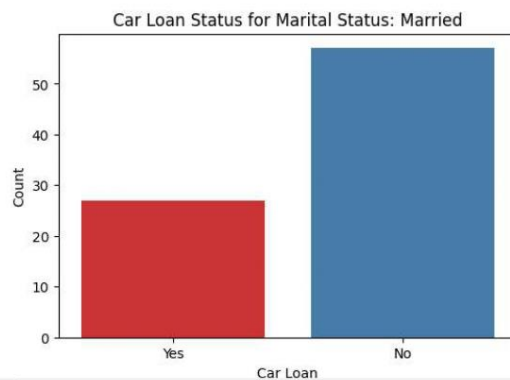
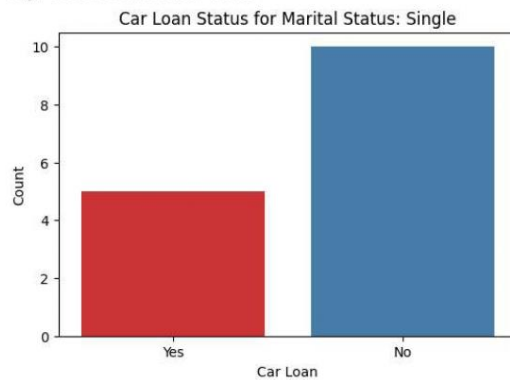
	Age	Profession	Marrital Status	Education	No of Dependents	Car_Loan	Total Salary	EV_Price
0	27	Salaried	Single	Post Graduate	0	Yes	800000	800000
1	35	Salaried	Married	Post Graduate	2	Yes	2000000	1000000
2	45	Business	Married	Graduate	4	Yes	1800000	1200000
3	41	Business	Married	Post Graduate	3	No	2200000	1200000
4	31	Salaried	Married	Post Graduate	2	Yes	2600000	1600000

Count Plot for Marital Status and Car Loan

```
# List unique marital statuses
marital_statuses = data['Marrital Status'].unique()

# Plot count plots for each marital status category
plt.figure(figsize=(10, 6))
for status in marital_statuses:
    plt.figure(figsize=(6, 4))
    sns.countplot(x='Car_Loan', data=data[data['Marrital Status'] == status], palette='Set1')
    plt.title(f'Car Loan Status for Marital Status: {status}')
    plt.xlabel('Car Loan')
    plt.ylabel('Count')
    plt.show()
```

 <Figure size 1000x600 with 0 Axes>



✓ Doughnut Chart for Car Loan Status

```
# Getting labels and data
labels = ['Car Loan Required', 'Car Loan not required']
loan_status = [data.query('Car_Loan == "Yes"').Car_Loan.count(), data.query('Car_Loan == "No"').Car_Loan.count()]

# Define explode for doughnut effect
explode = [0.1, 0]

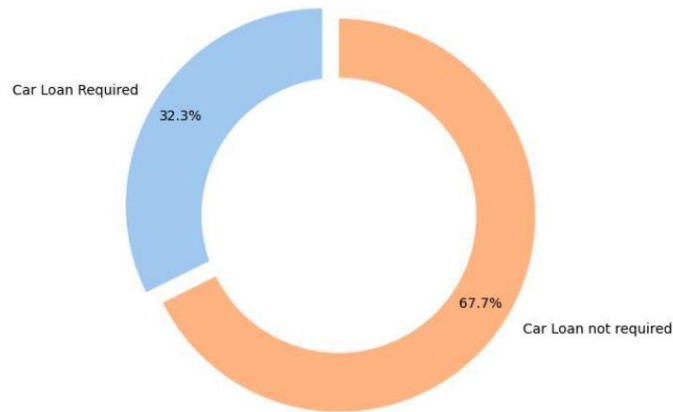
# Define color palette
palette_color = sns.color_palette('pastel')

# Plotting data on doughnut chart
fig, ax = plt.subplots()
ax.pie(loan_status, labels=labels, colors=palette_color, autopct='%1.1f%%', startangle=90, pctdistance=0.85, explode=explode)

# Draw a circle at the center to make it a doughnut chart
center_circle = plt.Circle((0, 0), 0.70, fc='white')
fig.gca().add_artist(center_circle)

# Equal aspect ratio ensures that pie is drawn as a circle
ax.axis('equal')

# Displaying the doughnut chart
plt.tight_layout()
plt.show()
```



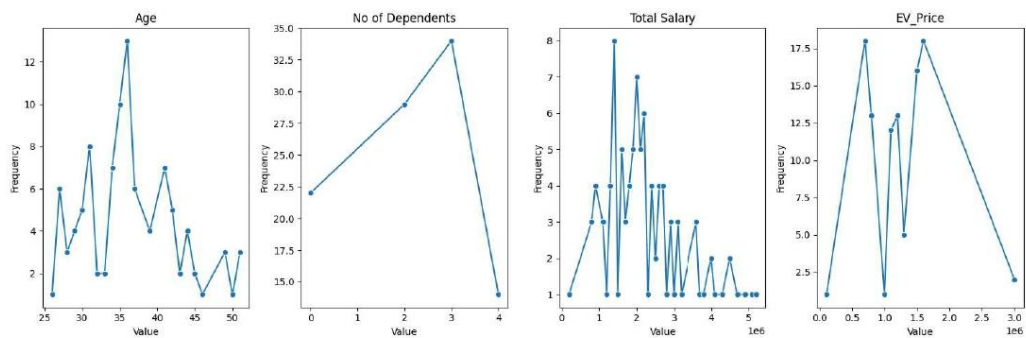
Line Charts for Consumer Features

```
# Plotting frequency of each entry for consumer features with line charts
plt.figure(figsize=(15, 5))

# List of consumer features
consumer_features = ['Age', 'No of Dependents', 'Total Salary', 'EV_Price']

# Iterate through each feature and plot line chart
for i, feature in enumerate(consumer_features, 1):
    plt.subplot(1, 4, i)
    plt.subplots_adjust(hspace=0.5, wspace=0.5)
    sns.lineplot(data=data[feature].value_counts().sort_index(), marker='o')
    plt.title(feature)
    plt.xlabel('Value')
    plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



✓ Step 9: K-Prototypes Clustering

Apply K-Prototypes clustering to the behavioral dataset for customer segmentation.

```
!pip install kmodes
from kmodes.kprototypes import KPrototypes

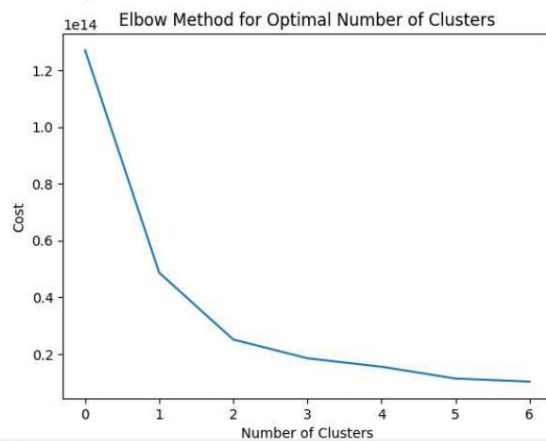
# Display the first few rows of the dataset
data.head()

# Define the features for clustering
cluster_features = list(data.columns)
cluster_data = data[cluster_features].values

# Identify categorical features (adjust indices as needed)
categorical_features = [1, 2, 3, 5] # Indices of categorical features

# Finding the optimal number of clusters using the Elbow Method
cost = []
for num_clusters in range(1, 8):
    kproto = KPrototypes(n_clusters=num_clusters, init='Cao')
    kproto.fit_predict(cluster_data, categorical=categorical_features)
    cost.append(kproto.cost_)
plt.plot(cost)
plt.title("Elbow Method for Optimal Number of Clusters")
plt.xlabel("Number of Clusters")
plt.ylabel("Cost")
plt.show()
```

```
Collecting kmodes
  Downloading kmodes-0.12.2-py2.py3-none-any.whl.metadata (8.1 kB)
Requirement already satisfied: numpy>=1.10.4 in /usr/local/lib/python3.10/dist-packages (from kmodes) (1.26.4)
Requirement already satisfied: scikit-learn>=0.22.0 in /usr/local/lib/python3.10/dist-packages (from kmodes) (1.3.2)
Requirement already satisfied: scipy>=0.13.3 in /usr/local/lib/python3.10/dist-packages (from kmodes) (1.13.1)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-packages (from kmodes) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22.0->kmodes) (3.5.0)
Downloading kmodes-0.12.2-py2.py3-none-any.whl (20 kB)
Installing collected packages: kmodes
Successfully installed kmodes-0.12.2
```



✓ Segregate Each Cluster

After determining the optimal number of clusters, we segregate the data into different clusters.

✓ Analyze EV Price vs Other Factors

Compare the average EV price and its relationship with other factors like salary, age, and number of dependents across clusters.

```
# Display the mean EV price
print(f"Average EV Price: {data.EV_Price.mean()}")

# Plot the relationship between EV Price and Age for each cluster
plt.scatter(Cluster_0.EV_Price, Cluster_0['Age'], color='red', marker='x', label='Cluster 1')
plt.scatter(Cluster_1.EV_Price, Cluster_1['Age'], color='green', label='Cluster 2')
plt.legend(loc="upper left")
plt.xlabel('EV Price')
plt.ylabel('Age')
plt.title('EV Price vs Age by Cluster')
plt.show()

# Plot the relationship between EV Price and Total Salary for each cluster
plt.scatter(Cluster_0.EV_Price, Cluster_0['Total Salary'], color='red', marker='x', label='Cluster 1')
plt.scatter(Cluster_1.EV_Price, Cluster_1['Total Salary'], color='green', label='Cluster 2')
plt.legend(loc="upper left")
plt.xlabel('EV Price')
plt.ylabel('Total Salary')
plt.title('EV Price vs Total Salary by Cluster')
plt.show()

# Plot the relationship between EV Price and Number of Dependents for each cluster
plt.scatter(Cluster_0.EV_Price, Cluster_0['No of Dependents'], color='red', marker='x', label='Cluster 1')
plt.scatter(Cluster_1.EV_Price, Cluster_1['No of Dependents'], color='green', label='Cluster 2')
plt.legend(loc="upper left")
plt.xlabel('EV Price')
plt.ylabel('Number of Dependents')
plt.title('EV Price vs Number of Dependents by Cluster')
plt.show()
```

Average EV Price: 1154042.4040404041

